Viridis Research — certification record

Run `runs/2026-11-05_P3` 2026-11-05 project P3, task P3-T001 engine `80671cff27c3`
GitHub run: `https://github.com/jdhart81/viridis-engine/actions/runs/41940722827`

Proposed Sealed Statements for a Circular-Economy Kernel: Loop Mass Balance, Cascaded Recovery, a Recovery-Adjusted Work Floor, and the N-Partner Symbiosis Condition
Justin Hart, Viridis LLC

Labels awarded by the verifier

Claim	Label	Claim text (as submitted)
C1	**ASSUMED**	empirical: Each of the four proposal modules compiles under the pinned toolchain, and none of the tactics simp, norm_num, decide, aesop, trivial, simp_all or exact? closes its Statement.

Other checks

- Lean toolchain leanprover/lean4:v4.28.0, Mathlib 8f9d9cff6bd7; kernel replay by `leanchecker`.
- References: 6 entries, 6 DOIs, 6 resolved, 0 unresolved, 0 unchecked.
- Agent paper: compiled.
- Statement proposals: 4.
- P3.LoopMassBalance: compiled; no probe tactic closes it; submitted for sealing
- P3.CascadedRecovery: compiled; no probe tactic closes it; submitted for sealing
- P3.RecoveryWorkFloor: compiled; no probe tactic closes it; submitted for sealing
- P3.CyclicSymbiosis: compiled; no probe tactic closes it; submitted for sealing

How to read this record

ASSUMED: not verified (the default). **PROVEN**: a Lean 4 proof of a statement sealed before the attempt, compiled with the pinned toolchain, matched definitionally to the sealed statement, using only the axioms propext, Classical.choice and Quot.sound, and replayed through the kernel. **TESTED/REPLICATED**: reproduced by the verifier (not yet enabled). **ENDORSED/ADOPTED**: granted by people. Novelty is never a label. **TRIVIAL** marks a sealed statement that one of the engine's probe tactics already closes.

This paper was drafted by an AI research agent (Claude, by Anthropic) running the Viridis research engine; formal proof search may use Aristotle (Harmonic). Justin Hart directs the research program. The labels on this page were assigned by an automated verifier, not by the agent, and only they carry certification. Claims in the paper that are not listed here as PROVEN are ASSUMED.

Reproduce everything from `REPRODUCE.md` in the run folder; the run's files are listed with hashes in `MANIFEST.sha256`.

Proposed Sealed Statements for a Circular-Economy Kernel: Loop Mass Balance, Cascaded Recovery, a Recovery-Adjusted Work Floor, and the N-Partner Symbiosis Condition

Justin Hart, Viridis LLC

5 November 2026

Abstract

A software kernel for the circular economy has to do a few pieces of arithmetic correctly every time. It must split demand into virgin and recovered input, track a secondary stock, compose recovery stages, bound the fresh work needed when recovered supply is available, and decide when a ring of firms exchanging by-products can leave every member strictly better off. We propose four formal target statements for this arithmetic, written in Lean 4 over Mathlib, as candidates for sealing in the Viridis Research program. *LoopMassBalance* gives the virgin/recovered split, an exact geometric closed form and the convergence of the secondary stock, and strict monotonicity of cumulative virgin input in the recovery fraction. *CascadedRecovery* bounds the virgin share $\prod_i (1 - r_i)$ after n stages from below by $(1 - \max_i r_i)^n$ and from above by every single stage's share, strictly so under a stated condition. *RecoveryWorkFloor* recasts a canon work-ledger bound as a floor on fresh supply given recovered supply and shows that the floor is attained. *CyclicSymbiosis* states that a strictly positive by-product flow around an N-partner cycle giving every partner strictly positive net benefit exists if and only if $\prod_i c_i < \prod_i v_i$. Its $N = 2$ case coincides with the product condition of the two-partner Reciprocity Corridor. Every module compiles under the pinned toolchain and carries concrete sanity checks and negative controls. Simple automated tactics close none of the four statements. We make no claim to have proved any proposal. The statements fix the mathematics of a model and say nothing about any real recovery rate, cost, value or material.

1 Introduction

Circular-economy accounting rests on simple identities and inequalities. Material that is recovered does not have to be extracted. Recovery stages in series compound. Recovered energy or work offsets fresh supply. A ring of firms exchanging by-products is viable only if the exchange creates value overall [3, 1]. These statements are elementary, but a decision kernel that applies them automatically should rely on exact, machine-checked versions with explicitly listed hypotheses, not on informal folklore. Edge cases are where silent errors creep in. Examples are full stock retention, out-of-range recovery fractions, stages that recover nothing, and partners whose own exchange is unprofitable but whose cycle is profitable.

Viridis Research separates *proposing* a formal target from *proving* it. A proposal is a Lean module that defines a proposition `Statement`. If the module compiles and simple tactics do not close it, the program owner decides whether to seal it as an exact target for later proof sessions. This paper is the first entry of project P3 ("Circular-economy kernel"). It drafts four such modules

and explains the design choices behind them. The question we address is: *what exact statements should the circular-economy kernel's formal core consist of?* Each statement must be faithful to the intended model, strong enough to be useful, provably non-vacuous, and continuous with the two canon results it builds on.

Contributions.

- Four self-contained Lean 4 statement modules over Mathlib [2, 6] (Section 4), each with sanity witnesses and negative controls checked at compile time.
- A recovery-adjusted reading of the canon work-ledger bound [5], with an attainment half that the canon lacks.
- An N-partner generalisation of the canon Reciprocity Corridor's feasibility theorem [4], together with a compile-time check that its $N = 2$ instance reproduces the canon's `ProductCondition` and payoffs.
- An automated probe (Section 5) showing that none of the four statements is closed by `simp`, `norm_num`, `decide`, `aesop`, `trivial`, `simp_all` or `exact?`.

2 Background and related work

Circular economy and industrial symbiosis. Geissdoerfer et al. [3] review how the circular-economy concept relates to sustainability. Its operational core is closing material loops, which means substituting recovered for virgin material. Chertow [1] defined industrial symbiosis as the exchange of materials, energy, water and by-products among co-located firms and gave a taxonomy of exchange types. Our *CyclicSymbiosis* model is a deliberately stylised, linear version of one such exchange pattern: a single directed ring of by-product transfers.

Canon results. The Viridis canon contains two results that P3 builds on. The Reciprocity Corridor (Run-128) [4] studies two partners X, Y with payoffs $u_X = v_X y - c_X x$ and $u_Y = v_Y x - c_Y y$. Its Lean formalisation proves that strictly positive flows with $u_X, u_Y > 0$ exist if and only if $c_X c_Y < v_X v_Y$ (`ProductCondition`). The Useful and Dissipated Work Ledger (Run-163) [5] defines useful work $e_1 w_1 + e_2 w_2$ from two supplies and proves `target_requires_supply`: if every efficiency is at most $e_{\max} > 0$ and useful work reaches T, then $w_1 + w_2 \geq T/e_{\max}$.

Formal tools. All modules are written for Lean 4 [2] (toolchain v4.28.0) and Mathlib [6] (commit `8f9d9cff`). They use only standard Mathlib notions: finite products and sums, `Filter.Tendsto`, `StrictAntiOn` and `iSup` on $\mathbb{R}$.

3 Design principles

1. **Self-contained.** Each module imports only Mathlib, so a sealed statement does not depend on canon files that might change. Where a canon definition is needed for comparison (`ProductCondition`), it is copied verbatim.
2. **All hypotheses explicit.** Each hypothesis appears in the proposition as an implication. Nothing is hidden in a structure or type class.

3. **Faithful to the brief, slightly stronger where cheap.** For example, LoopMassBalance asserts the exact closed form for the gap $S(t) - S^*$ as well as convergence, and CascadedRecovery adds a uniform-upper-bound version of the max bound.

4. **Controls compiled with the module.** Every module contains `example`s with concrete numbers, proved by `norm_num`, `simp` or `decide`. They include *witnesses*, which show that the hypotheses are satisfiable and the conclusions non-trivial, and *negative controls*, which show that dropping a hypothesis breaks a conclusion.

5. **No field claims.** The statements are about models. Recovery rates, retention rates, efficiencies, costs and values are free real parameters.

4 The four proposed statements

4.1 P3.LoopMassBalance

Proposed statement 1 (Loop mass balance)**.** *Let $D \geq 0$, $r \in [0,1]$, $\rho \in [0,1)$ and $S(0) \in \mathbb{R}$. Put $V = D(1-r)$, $S(t+1) = \rho S(t) + rD$ and $S^* = rD/(1-\rho)$. Then $0 \leq V \leq D$; $S(t) - S^* = \rho^t(S(0) - S^*)$ for all $t \in \mathbb{N}$; $S(t) \to S^*$; and $\sum_{t<T} V = TD(1-r)$. Moreover, if $D > 0$ and $T > 0$, then $r \mapsto \sum_{t<T} V$ is strictly decreasing on $[0,1]$.*

Informal argument. The gap $g_t = S(t) - S^*$ satisfies $g_{t+1} = \rho g_t$ because $\rho S^* + rD = S^*$. Since $0 \leq \rho < 1$, $\rho^t \to 0$. The remaining parts are elementary. (No Lean proof is claimed here.)

Listing 1: Core of `proposals/P3.LoopMassBalance.lean` (docstrings omitted; `nhds` is the name behind Mathlib's $\mathcal{N}$ notation used in the file).

```
namespace ViridisStatements.P3.LoopMassBalance

open Filter Topology

def virgin (D r : ℝ) : ℝ := D * (1 - r)

def stock (ρ r D S0 : ℝ) : ℕ → ℝ
  | 0 => S0
  | t + 1 => ρ * stock ρ r D S0 t + r * D

def steadyStock (ρ r D : ℝ) : ℝ := r * D / (1 - ρ)

def cumulativeVirgin (D r : ℝ) (T : ℕ) : ℝ := ∑ _t ∈ Finset.range T, virgin D r

def Statement : Prop :=
  (∀ D r ρ S0 : ℝ, 0 ≤ D → 0 ≤ r → r ≤ 1 → 0 ≤ ρ → ρ < 1 →
    (0 ≤ virgin D r ∧ virgin D r ≤ D) ∧
    (∀ t : ℕ, stock ρ r D S0 t - steadyStock ρ r D
        = ρ ^ t * (S0 - steadyStock ρ r D)) ∧
    Tendsto (stock ρ r D S0) atTop (nhds (steadyStock ρ r D)) ∧
    (∀ T : ℕ, cumulativeVirgin D r T = (T : ℝ) * D * (1 - r))) ∧
  (∀ (D : ℝ) (T : ℕ), 0 < D → 0 < T →
    StrictAntiOn (fun r => cumulativeVirgin D r T) (Set.Icc 0 1))
```

Controls. The witness $D = 10$, $r = \rho = \frac{1}{2}$, $S(0) = 0$ gives $V = 5$, the stock sequence $0, 5, \frac{15}{2}, \ldots$ and $S^* = 10$, and the gap at $t = 2$ equals $(\frac{1}{2})^2(0-10)$. Over three periods, cumulative virgin input falls from 15 to $\frac{15}{2}$ when r rises to $\frac{3}{4}$. There are three negative controls. With $\rho = 1$ the stock grows without bound and the closed form fails; Lean's convention $x/0 = 0$ makes $S^* = 0$. With $D = 0$ the cumulative virgin input does not depend on r. With $r = 2$ the virgin input is negative.

4.2 P3.CascadedRecovery

Proposed statement 2 (Cascaded recovery)**.** *Let $n \in \mathbb{N}$ and $r_1, \ldots, r_n \in [0,1]$, and let $\sigma = \prod_i (1 - r_i)$ be the virgin share. Then $0 \le \sigma \le 1$, and $(1-m)^n \le \sigma$ for every $m \le 1$ with $r_i \le m$ for all i. In particular $(1 - \sup_i r_i)^n \le \sigma$. Also $\sigma \le 1 - r_j$ for every j, with strict inequality whenever $r_j < 1$ and $r_k > 0$ for some $k \ne j$.*

Informal argument. Each factor lies in $[1-m, 1] \subseteq [0,1]$. Splitting off factor j gives $\sigma = (1-r_j)\prod_{i \ne j}(1-r_i)$, and the remaining product is at most $1 - r_k < 1$. The maximum is written as Mathlib's $\bigsqcup_i r_i$ (`iSup`). On $\mathrm{Fin}\,0$ this is $\sup \emptyset = 0$, and both sides then equal 1, so no side condition $n > 0$ is needed.

Listing 2: Core of `proposals/P3.CascadedRecovery.lean`.

```
namespace ViridisStatements.P3.CascadedRecovery

def virginShare {n : ℕ} (r : Fin n → ℝ) : ℝ := ∏ i, (1 - r i)

def Statement : Prop :=
  ∀ (n : ℕ) (r : Fin n → ℝ), (∀ i, 0 ≤ r i) → (∀ i, r i ≤ 1) →
    (0 ≤ virginShare r ∧ virginShare r ≤ 1) ∧
    (∀ m : ℝ, (∀ i, r i ≤ m) → m ≤ 1 → (1 - m) ^ n ≤ virginShare r) ∧
    (1 - ⨆ i, r i) ^ n ≤ virginShare r ∧
    (∀ j, virginShare r ≤ 1 - r j) ∧
    (∀ j k, k ≠ j → 0 < r k → r j < 1 → virginShare r < 1 - r j)
```

Controls. With $r = (\frac{1}{2}, \frac{1}{2})$ the share is $\frac{1}{4}$, which meets the lower bound and lies strictly below $\frac{1}{2}$. With $r = (\frac{1}{2}, \frac{1}{4}, 0)$ the share is $\frac{3}{8} > \frac{1}{8}$. Four negative controls show which hypotheses matter. With $r = (0, \frac{1}{2})$ the cascade equals stage 2, so strictness needs another stage with $r_k > 0$. With $r = (\frac{1}{2}, 1)$ the share is $0 = 1 - r_2$, so strictness needs $r_j < 1$. With $r = (2, \frac{1}{2})$ the share is negative. With $r = (-1, \frac{1}{2})$ the share exceeds $1 - r_2$.

4.3 P3.RecoveryWorkFloor

Proposed statement 3 (Recovery-adjusted work floor)**.** *Let $w_f, w_r \ge 0$, $e_f, e_r \in [0, e_{\max}]$ with $e_{\max} > 0$, and $U = e_f w_f + e_r w_r$. If $U \ge T$, then $w_f \ge (T - e_{\max} w_r)/e_{\max}$. Conversely, if $e_{\max} w_r \le T$, then $w_f^* = (T - e_{\max} w_r)/e_{\max}$ is non-negative and, at $e_f = e_r = e_{\max}$, delivers exactly T.*

Informal argument. We have $T \le U \le e_{\max}(w_f + w_r)$. Dividing by $e_{\max}$ and rearranging gives the floor. Attainment is a direct calculation.

Remark. Since $(T - e_{\max} w_r)/e_{\max} = T/e_{\max} - w_r$, the floor half is algebraically equivalent to Run-163's `target_requires_supply` specialised to a fresh and a recovered supply. The proposal is therefore a conservative extension. What it adds is the recovery reading (one unit of recovered supply at the cap offsets one unit of fresh supply) and the attainment half, which shows that the floor cannot be raised. Separate caps for fresh and recovered efficiencies would be a genuinely stronger alternative, and we flag it for the sealing decision.

Listing 3: Core of `proposals/P3.RecoveryWorkFloor.lean`.

```
namespace ViridisStatements.P3.RecoveryWorkFloor

def useful (wf wr ef er : ℝ) : ℝ := ef * wf + er * wr
```

```
def freshFloor (T wr emax : ℝ) : ℝ := (T - emax * wr) / emax

def Statement : Prop :=
  (∀ wf wr ef er emax T : ℝ, 0 ≤ wf → 0 ≤ wr →
    0 ≤ ef → ef ≤ emax → 0 ≤ er → er ≤ emax → 0 < emax →
    T ≤ useful wf wr ef er → freshFloor T wr emax ≤ wf) ∧
  (∀ wr emax T : ℝ, 0 < emax → 0 ≤ wr → emax * wr ≤ T →
    0 ≤ freshFloor T wr emax ∧ useful (freshFloor T wr emax) wr emax emax = T)
```

Controls. The Run-163 witness $(8, 4, \frac{3}{4}, \frac{1}{2})$ gives $U = 8$ and floor $\frac{20}{3} \leq 8$. For attainment, $e_{\max} = \frac{1}{2}$, $w_r = 4$ and $T = 8$ give floor 12, and 12 units of fresh supply deliver exactly 8. Without recovery the floor would be 16. There are three negative controls. Fresh supply $11 < 12$ falls short of the target. An efficiency of 1 above the cap reaches the target below the floor. With $e_{\max} w_r > T$ the floor is negative.

4.4 P3.CyclicSymbiosis

Proposed statement 4 (N-partner cyclic symbiosis)**.** *Let $N \geq 2$ and $c_i, v_i > 0$ for $i \in \mathbb{Z}/N$. Partner i ships x_i at cost $c_i x_i$ and receives value $v_i x_{i-1}$. A flow x with all $x_i > 0$ and all net benefits $v_i x_{i-1} - c_i x_i > 0$ exists if and only if $\prod_i c_i < \prod_i v_i$.*

Informal argument. ($\Rightarrow$) Multiply the N strict inequalities $c_i x_i < v_i x_{i-1}$ between positive numbers. Index shift is a bijection, so $\prod_i x_{i-1} = \prod_i x_i > 0$ cancels. ($\Leftarrow$) Put $a_i = v_i/c_i$, $P = \prod a_i > 1$ and $\lambda = P^{1/N} > 1$. Set $x_0 = 1$ and $x_i = x_{i-1} a_i / \lambda$ for $1 \leq i < N$. Then $x_i < a_i x_{i-1}$ for $i \geq 1$, and $a_0 x_{N-1} = P/\lambda^{N-1} = \lambda > 1 = x_0$ closes the cycle.

Listing 4: Core of `proposals/P3.CyclicSymbiosis.lean`. `ProductCondition` is copied verbatim from the canon file.

```
namespace ViridisStatements.P3.CyclicSymbiosis

def prev {N : ℕ} (i : Fin N) : Fin N :=
  ⟨(i.val + (N - 1)) % N, Nat.mod_lt _ (Nat.zero_lt_of_lt i.isLt)⟩

def net {N : ℕ} (c v x : Fin N → ℝ) (i : Fin N) : ℝ := v i * x (prev i) - c i * x i

def StrictlyBeneficialFlow {N : ℕ} (c v x : Fin N → ℝ) : Prop :=
  (∀ i, 0 < x i) ∧ ∀ i, 0 < net c v x i

def CycleProductCondition {N : ℕ} (c v : Fin N → ℝ) : Prop := ∏ i, c i < ∏ i, v i

def ProductCondition (vX cX vY cY : ℝ) : Prop := cX * cY < vX * vY

def Statement : Prop :=
  ∀ (N : ℕ), 2 ≤ N → ∀ c v : Fin N → ℝ, (∀ i, 0 < c i) → (∀ i, 0 < v i) →
    ((∃ x : Fin N → ℝ, StrictlyBeneficialFlow c v x) ↔ CycleProductCondition c v)
```

Agreement with the canon at $N = 2$. The module checks two facts at compile time. First, `CycleProductCondition ![cX, cY] ![vX, vY]` is equivalent to Run-128's `ProductCondition vX cX vY cY`. Second, at the flow `![x, y]` partner 0 nets $v_X y - c_X x = u_X$ and partner 1 nets $v_Y x - c_Y y = u_Y$. Hence the $N = 2$ instance of the statement is the canon's strict-feasibility theorem in its bilateral-benefit form.

Listing 5: The $N = 2$ agreement checks, as they appear in the module.

```
example (vX cX vY cY : ℝ) :
    CycleProductCondition ![cX, cY] ![vX, vY] ↔ ProductCondition vX cX vY cY := by
  simp [CycleProductCondition, ProductCondition, Fin.prod_univ_two]

example (vX cX vY cY x y : ℝ) :
    net ![cX, cY] ![vX, vY] ![x, y] 0 = vX * y - cX * x ∧
    net ![cX, cY] ![vX, vY] ![x, y] 1 = vY * x - cY * y := by
  have h0 : prev (0 : Fin 2) = 1 := by decide
  have h1 : prev (1 : Fin 2) = 0 := by decide
  simp [net, h0, h1]
```

Controls. The values of `prev` on $\mathrm{Fin}\,3$ and $\mathrm{Fin}\,2$ are checked by `decide`. The three-partner economy $c = (1, 1, 1)$, $v = (4, \frac{1}{2}, 1)$ satisfies $1 < 2$ even though partner 1 alone has $v_1 < c_1$. The uniform flow gives partner 1 a negative net, while $x = (1, \frac{9}{20}, \frac{2}{5})$ gives nets $(\frac{3}{5}, \frac{1}{20}, \frac{1}{20})$, all positive. The flow must therefore be tuned to the cycle. There are two negative controls. $c = (2, 2, 2)$, $v = (1, 1, 1)$ fails the condition, and the uniform flow harms partner 0. The boundary case $c = v = (1, 1)$ fails the strict condition.

5 Results

All four modules compile under the pinned toolchain without errors or warnings, including every sanity example. We probed each `Statement` with seven simple tactics (`code/probe.sh`, output in `results/probe_results.tsv`). Table 1 summarises the outcome. No tactic closed any statement, so none is trivial in the sense the verifier tests. Table 2 lists the hypotheses of each statement. These results describe the proposals only. Proofs belong to later tasks, after a sealing decision.

Statement	`simp`	`norm_num`	`decide`	`aesop`	`trivial`	`simp_all`	`exact?`
LoopMassBalance	open	open	open	open	open	open	open
CascadedRecovery	open	open	open	open	open	open	open
RecoveryWorkFloor	open	open	open	open	open	open	open
CyclicSymbiosis	open	open	open	open	open	open	open

Table 1: Probe outcomes. "open" means the tactic failed to close the statement.

Statement	Hypotheses
LoopMassBalance	$0 \le D$, $0 \le r \le 1$, $0 \le \rho < 1$ (parts 1–4); $0 < D$, $0 < T$, $r \in [0, 1]$ (strict monotonicity)
CascadedRecovery	$0 \le r_i \le 1$ for all i; $r_i \le m \le 1$ (uniform bound); $k \ne j$, $r_k > 0$, $r_j < 1$ (strictness)
RecoveryWorkFloor	$w_f, w_r \ge 0$, $0 \le e_f, e_r \le e_{\max}$, $e_{\max} > 0$, $T \le U$ (floor); $e_{\max} > 0$, $w_r \ge 0$, $e_{\max} w_r \le T$ (attainment)
CyclicSymbiosis	$N \ge 2$, $c_i > 0$, $v_i > 0$ for all i

Table 2: Hypotheses of each proposed statement.

6 Limitations

- **Models, not measurements.** No statement says anything about real recovery rates, retention, efficiencies, costs, values, materials or environmental outcomes. Those remain field inputs.
- **Stylised dynamics.** LoopMassBalance has constant demand and rates, a single material and no feedback from stock to recovery. CascadedRecovery treats stages as independent. CyclicSymbiosis covers a single directed cycle with linear payoffs and no capacities, transport losses or division of surplus. Run-128's Nash-midpoint result has no N-partner analogue here.
- **Mild extension in RecoveryWorkFloor.** Its floor half is a rearrangement of the canon bound (see the remark above). It is not a thermodynamic bound: it is neither a Landauer bound nor dependent on the Intelligence Bound, which remains a conjecture.
- **Probes are not proofs of hardness.** Failing seven simple tactics does not show that a statement is deep. It only shows that the statement is not closed by those tactics.
- **Tooling.** An attempt to use the Aristotle prover failed at submission with an authentication error. No Aristotle project was created.

7 Reproducibility

Each file `proposals/P3.<ID>.lean` compiles with `lean` under toolchain `leanprover/lean4:v4.28.0` and Mathlib commit `8f9d9cff6bd728b17a24e163c9402775d9e6a365`. The probe table is regenerated by `code/probe.sh proposals <scratch-dir>`. The rationale files `proposals/P3.<ID>.md` list every hypothesis, every control and what each statement does not settle. The two canon files used for comparison are copied from the canon repository at commit `17a0702f`, and their hashes are recorded by the project.

8 Changelog

- 2026-11-05 (P3-T001): first version. Proposes P3.LoopMassBalance, P3.CascadedRecovery, P3.RecoveryWorkFloor and P3.CyclicSymbiosis, with controls, probe results and rationale. No prior P3 paper existed.

References

[1] Marian R. Chertow. Industrial symbiosis: Literature and taxonomy. *Annual Review of Energy and the Environment*, 25:313–337, 2000. doi: 10.1146/annurev.energy.25.1.313.

[2] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[3] Martin Geissdoerfer, Paulo Savaget, Nancy M. P. Bocken, and Erik Jan Hultink. The circular economy – a new sustainability paradigm? *Journal of Cleaner Production*, 143:757–768, 2017. doi: 10.1016/j.jclepro.2016.12.048.

[4] Justin D. Hart. The reciprocity corridor: A bilateral-benefit test for directional information exchange. Zenodo working paper (Viridis Run-128), not peer reviewed, 2026.

[5] Justin D. Hart. A useful-dissipated work ledger note for staged ecopoiesis. Zenodo preprint (Viridis Run-163), not peer reviewed, 2026.

[6] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.